SANDIP UNIVERSITY
UGC Recognised
NAAC GRADE A

CLOUD IAM ENTITLEMENT AUDIT

Detecting Exposed Cloud Storage, Broad Permissions, and Hard-Coded Secrets in Infrastructure-as-Code

Project Report

Project objective: develop a lightweight Python utility to identify common cloud security misconfigurations in local Terraform/HCL and JSON configuration files. The project also documents how Checkov and Terraform validation can complement the custom scanner.

Abstract

Cloud infrastructure is frequently defined through code. Misconfigured storage access, overly broad IAM policies, and secrets committed to configuration repositories can expose sensitive data or enable unauthorized actions. This project provides a simple, repeatable static scan that highlights selected risky patterns, prints actionable recommendations, and can export findings to JSON for review.

1. Problem statement

Manual review of infrastructure-as-code can miss risky configuration details, especially across multiple files. The project aims to provide a fast first-pass audit that helps reviewers focus on potentially public storage, wildcard permissions, public database settings, and embedded credential material.

2. Objectives

- Scan Terraform/HCL, variable, and JSON files locally.
- Flag selected public-access and IAM permission patterns.
- Identify possible hard-coded cloud credentials and private-key material.
- Provide severity labels, evidence, remediation guidance, and JSON output.
- Support a simple command-line workflow that can be used during code review.

3. System design and workflow

1) User supplies a file or directory. 2) The scanner discovers supported files while skipping common generated directories. 3) Each non-comment line is tested against a set of documented patterns. 4) Matching rules become findings with rule ID, severity, file, line, evidence, and recommendation. 5) Findings are printed and may be exported to JSON. 6) An optional severity threshold controls the process exit code.

4. Detection rules

Rule	Severity	Detection / review signal
CIA001	High	Storage ACL set to public-read or public-read-write
CIA002	High	Selected storage public-access protections explicitly disabled

Rule	Severity	Detection / review signal
CIA003	High	publicly_accessible = true
CIA004	High	IAM actions wildcard (*)
CIA005	High	IAM resources wildcard (*)
CIA006	High	Possible AWS access-key identifier
CIA007	Critical	Private-key header material
CIA008	High signal	IAM Allow statement; requires context review
CIA009	Medium	Empty versioning block review signal
CIA010	High	Possible hard-coded secret value

5. Implementation

The Python program uses only the standard library: argparse for command-line options, pathlib for file discovery, re for pattern matching, dataclasses for structured findings, and json for report export. Supported extensions are .tf, .tfvars, .hcl, and .json. The scanner is intentionally read-only.

6. Example usage

```
python cloud_iam_entitlement_audit.py ./terraform
python cloud_iam_entitlement_audit.py ./terraform --json audit-findings.json
python cloud_iam_entitlement_audit.py ./terraform --fail-on CRITICAL
```

Optional complementary checks

```
python -m pip install checkov
checkov -d ./terraform --framework terraform
terraform fmt -check -recursive
terraform validate
```

7. Expected output

The console summary reports the number of files scanned and findings identified. Each finding includes severity, rule ID, file path, line number, a short evidence excerpt, and a remediation recommendation. JSON output supports later filtering, archiving, or integration into a review process.

8. Security recommendations

• Block public storage access by default and document exceptions.
• Follow least privilege: restrict IAM actions, resources, and conditions.
• Never commit credentials or private keys; rotate any real secret found in source history.
• Prefer managed secret stores and workload identity over static keys.
• Validate findings with Checkov, Terraform validation, cloud-native posture tools, and manual policy review.

9. Limitations and future improvements

This scanner is heuristic and may produce false positives or miss risks expressed through modules, variables, dynamic blocks, remote state, policy documents, or provider-specific syntax. It does not query live cloud resources or determine effective permissions. Future enhancements could include structured HCL parsing, provider-specific rule packs, SARIF output, CI integration, and authenticated read-only cloud inventory checks.

10. Conclusion

The Cloud IAM Entitlement Audit project demonstrates a small, transparent approach to detecting common infrastructure-as-code risk patterns. Used as an early review aid alongside Checkov, Terraform validation, and organization-approved security controls, it can help teams identify issues sooner and improve configuration hygiene.

Disclaimer: This report describes a sample educational project implementation. It is not a certification of compliance or security and should be adapted and validated before production use.